



**BACHELOR OF COMPUTER SCIENCE
(SOFTWARE ENGINEERING) WITH HONOURS
SEMESTER II 2025/2026**

**CSE3433
SOFTWARE ARCHITECTURE**

**GROUP 13: TALENTFLOW
EVENT – DRIVEN JOB RECRUITMENT & APPLICATION
PROCESSING SYSTEM**

GROUP MEMBERS	MATRIC NO.
IZZ DHIA DAMIA BINTI MAT ZAIDI	S75849
NURUL IFFAH SABRINA BINTI AHMAD SHUKRI	S75894

TABLE OF CONTENT

1.0 Introduction.....	3
2.0 Problem Description.....	4
3.0 System Requirements.....	5
4.0 Architecture Design.....	8
5.0 Architecture Diagrams.....	9
6.0 Technology Stack.....	14
7.0 Prototype Implementation.....	15
8.0 Architecture Evaluation.....	17
9.0 Team Contributions.....	18
10.0 Conclusion and Reflection.....	19

1.0 INTRODUCTION

TalentFlow is a kind of innovative recruitment management system, its goal is to bring automation and more efficiency into the hiring process. In most conventional recruiting software, the system still stays very dependent on manual operations, like checking resumes, updating candidate status, and scheduling interviews. These manual steps feel cumbersome, take a lot of time, and are hard to manage, especially when there are many applicants all at once. To tackle those issues, TalentFlow uses the concept of Event-Driven Architecture (EDA). EDA helps the software respond quickly to significant events such as posting jobs, receiving applications, and changing statuses. Instead of waiting for someone to submit a direct request, the system reacts automatically whenever an event happens inside the platform, like a quiet but constant signal. The tool ends up serving two main groups of users, more or less which is candidates. They can browse job opportunities, apply online, get confirmation emails, and receive timely updates about where their application stands. Next is recruiters, they can publish job descriptions, assess candidate applications, update the recruitment status, short list candidates, and monitor overall recruitment progress through dashboards. With this setup, TalentFlow can operate effectively, react promptly, and scale up fairly easily while handling several applicants in parallel.

2.0 PROBLEM DESCRIPTION

In many organizations, recruitment still leans on mostly manual operations, and that really creates a bunch of obstacles that slow hiring way down and also makes the candidate experience kind of poor. The Human Resource (HR) team has to look at each resume one by one, then contact candidates by hand, so the whole process stays extremely slow. This means hiring decisions get delayed, and meanwhile good candidates can accept offers from competitors who move faster. Also, when people apply, nothing happens right away. They end up waiting for long periods with no clear response, no acknowledgement, and that poor communication makes candidates lose interest in the role. On top of that, the manual route brings a high likelihood of mistakes, like ignoring candidate details, misplacing applications, or simply not following up with the right people. And during peak recruitment times, the sheer volume of applicants becomes too much for HR staff, it piles up, causes burnout, and lowers the overall hiring quality. Because all these issues connect to each other, there is a strong need for an automated, event-driven recruitment system, one that can help streamline the process, reduce the hands-on workload, enhance how candidates are informed, and handle large batches of applications effectively.

3.0 SYSTEM REQUIREMENT

3.1 Functional Requirements

3.1.1 Job Management Module

FR-1.1: The system shall allow recruiters to create job posts by entering job title, description, requirements, and location.

FR-1.2: The system shall automatically update the job board when a new job post is created.

FR-1.3: The system shall allow candidates to view available job listings.

3.1.2 Application Management Module

FR-2.1: The system shall allow candidates to submit applications with their resumes and personal details.

FR-2.2: The system shall send a confirmation message automatically when a candidate submits an application.

FR-2.3: The system shall store and organize all applications for each job posting

3.1.3 Resume Filtering Module

FR-3.1: The system shall automatically filter resumes based on criteria such as skills, experience, and qualifications.

FR-3.2: The system shall categorize applicants as "Qualified" or "Not Qualified" based on filtering results.

FR-3.3: The system shall notify recruiters when resume screening is completed.

3.1.4 Status Management Module

FR-4.1: Recruiters shall be able to update candidate status (e.g., Shortlisted, Interviewed, Hired, Rejected).

FR-4.2: The system shall notify candidates immediately when their status is updated.

FR-4.3: The system shall maintain a history of status changes for each applicant.

3.1.5 Interview Management Module

FR-5.1: The system shall help schedule interviews and send invitations to candidates.

FR-5.2: Recruiters shall be able to view and manage scheduled interviews.

FR-5.3: The system shall send reminders to candidates before their scheduled interviews.

3.1.6 Dashboard and Reporting Module

FR-6.1: The system shall provide a dashboard showing real-time recruitment progress and statistics.

FR-6.2: The system shall display key metrics such as total applicants, shortlisted candidates, and hiring progress.

FR-6.3: Recruiters shall be able to generate reports on recruitment activities.

3.2 Non-functional Requirements

3.2.1 Performance

NFR-1: The system must process job applications and trigger notifications within 2 seconds of an event occurrence.

NFR-2: Webpages must load and render completely within less than 2 seconds under normal conditions.

3.2.2 Scalability

NFR-3: The system must support thousands of job applications efficiently and handle spikes of over 1,000 concurrent users during peak recruitment periods.

NFR-4: The event-driven architecture must allow horizontal scaling of individual services (e.g., Resume Filter Service) without affecting other components.

3.2.3 Reliability

NFR-5: No event shall be lost during system load or temporary failures. Messages must persist in queues until successfully processed.

NFR-6: If a service fails, messages must remain in the queue and be processed when the service recovers.

3.2.4 Availability

NFR-7: Core services must be available 99.9% of the time to ensure uninterrupted recruitment operations.

NFR-8: The system must maintain operational stability during peak user traffic times.

3.2.5 Maintainability

NFR-9: The system must be easy to update or add new modules without affecting existing functionality.

NFR-10: Loose coupling between services must allow independent development and deployment.

3.2.5 Security

NFR-11: User authentication and authorization must be implemented to protect sensitive recruitment data.

NFR-12: Candidate personal information and resumes must be stored securely with appropriate access controls.

4.0 ARCHITECTURE DESIGN

TalentFlow uses an Event Driven Architecture, kind of like a publish-subscribe communication model, where things send messages as events and other parts listen for them. This setup helps the whole system scale better, stay easier to maintain, and feel more responsive, because the components do not have to talk in a strict, synchronous way. Instead, they exchange those events asynchronously, so the flow is smoother and less tied up.

4.1 Event-Driven Architecture Overview

The system runs entirely on events, like everything important turns into an event and then the next parts just start running on their own, kind of. Each meaningful action causes an event, and that event automatically kicks off downstream work. You can see events such as job postings, application submissions, resume screening, status updates, and interview scheduling. For instance, when a candidate submits an application, an `ApplicationSubmitted` event shows up and it triggers resume filtering, notification delivery, and dashboard updates all at once, so manual delays, and the coordination bits basically disappear.

4.2 Core Components

TalentFlow is made up of seven separate services that kinda talk only through events, no direct calls. The Job Service is responsible for job postings and it puts out `JobPosted` events. The Application Service takes care of candidate submissions, then it publishes `ApplicationSubmitted` events, pretty much straight after. Next up Resume Filter Service screens resume automatically and generate `ResumeScreened` events. After that the Status Management Service follows the candidate progress and emits `StatusUpdated` events, even when things get a bit messy. The Interview Service is in charge of scheduling interviews and it publishes `InterviewScheduled` events. Finally, there's the Notification Service which takes care of communications by consuming a bunch of different events. On top of that the Dashboard Service delivers near real time analytics by consuming all events as read only projections, so it can show everything without changing anything.

4.3 Event Bus Implementation

RabbitMQ acts as message broker sort of, and it implements the event bus that links all services together. In practice, the broker makes sure events land safely via message persistence and consumer acknowledgments, plus the whole queue-based distribution. When events are published, they go to a fanout exchange, and then each consumer service keeps its own individual queue so processing stays independent. This layout also helps in situations where things go wrong, because even during system failures or temporary service outages, events don't just vanish.

4.4 Cloud Deployment Architecture

The system gets deployed through cloud-native stuff, basically Docker containers run and managed by Kubernetes so scaling is automated and the whole thing stays in control, you know. The frontend sits on Vercel, while the backend services are on Render, and the data layer uses MongoDB Atlas. A load balancer spreads the traffic across multiple service instances, and cloud storage takes care of the resume files, AWS SES then handles email notifications too, that way everything stays scalable available, and gives consistent performance when the recruitment season is hitting peak, like really peak.

5.0 ARCHITECTURE DIAGRAMS

5.1 Context Diagram



Diagram 1.0 Context Diagram

The context diagram shows how external users interact with the TalentFlow system at a high level. The main users are candidates and recruiters, who communicate directly with the system. **Candidates** use the system to view job opportunities, submit applications, receive confirmations, and track their application status. **Recruiters** use the system to post job openings, review applications, update candidate status, schedule interviews, and monitor recruitment progress. The TalentFlow system acts as the central system that processes all requests and coordinates interactions with external entities, including email/SMS notification services, job boards, and dashboard reporting.

5.2 Component/Service Architecture Diagram

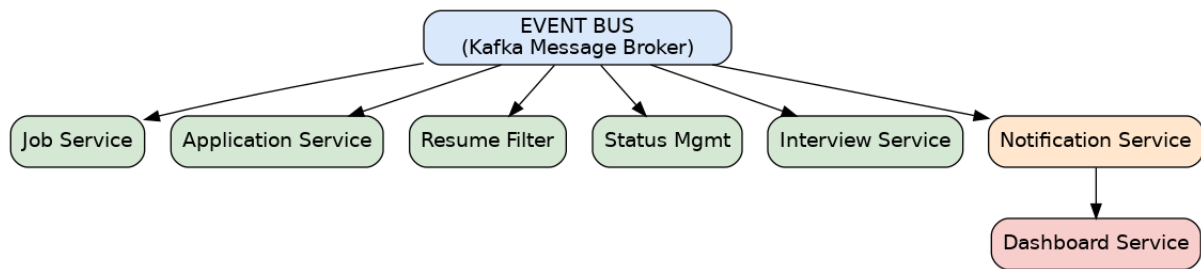


Diagram 1.2 Component/Service Architecture Diagram

The component and service architecture diagram sort of shows the internal shape of the TalentFlow system, like how the seven core services connect through the event bus, to create one aligned recruitment platform. In the middle of everything there is the Event Bus, built with RabbitMQ, and it acts as the communication spine that keeps all services detached from each other. Each service basically runs on its own publishing events to the bus, and it only consumes the ones that match its own scope. This removes tight coupling and makes it easier to build, deploy, and also scale the services separately. The Job Service and the Application Service are the main entry points, because they handle recruiter job postings and candidate application submissions, respectfully. Then the Resume Filter Service, the Status Management Service, the Interview Service, the Notification Service, and the Dashboard Service kind of work together, to process applications, steer candidate progress, arrange interview sessions, send out notifications, and offer live analytics. The modular setup matters here, because if something goes wrong, like the Resume Filter Service not being able to process applications, the rest of the services can keep running normally. Events will sit in the queue until that broken piece comes back, so the system stays reliable and available.

5.3 Interaction Diagrams

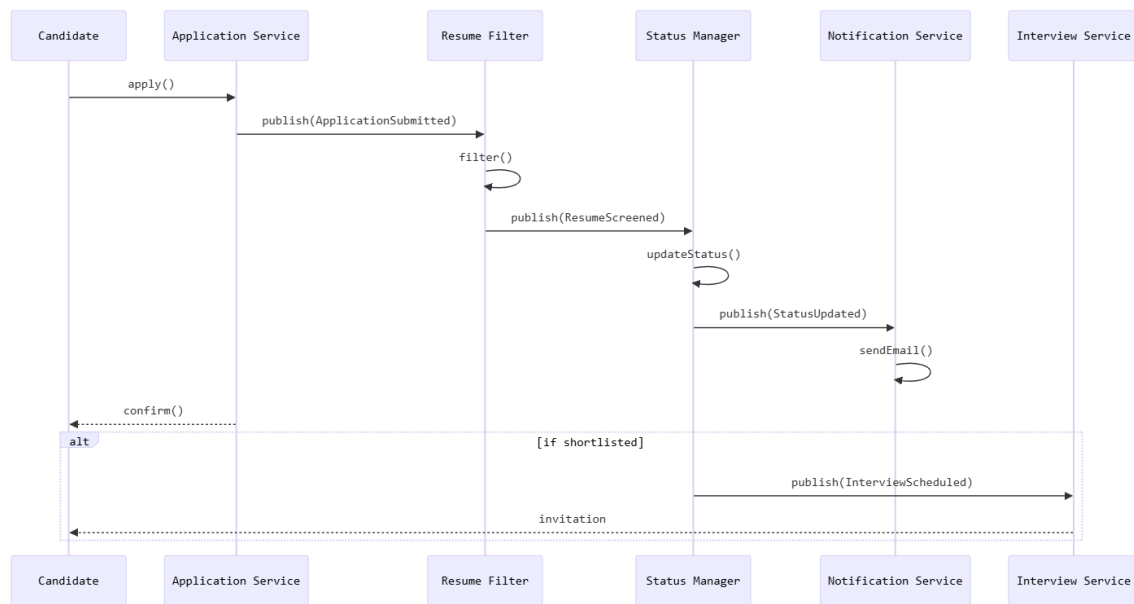


Diagram 1.3 Interaction Diagram

The diagram shows the complete flow of how the job application process works. The process starts with the applicant sending their application via the Application Service, which then triggers the event Application Submitted. This event will be handled by the Resume Filter service, whereby the applicant's resume will be reviewed. Thereafter, the event Resume Screened will be fired, with the Status Manager service updating the status of the applicant's application. An event Status Updated will follow suit, leading to the Notification Service sending an email to the applicant. If the applicant's resume qualifies, there will be an additional flow where an event Interview Scheduled will be triggered, and the applicant will receive an invite for an interview.

5.4 Deployment Diagram

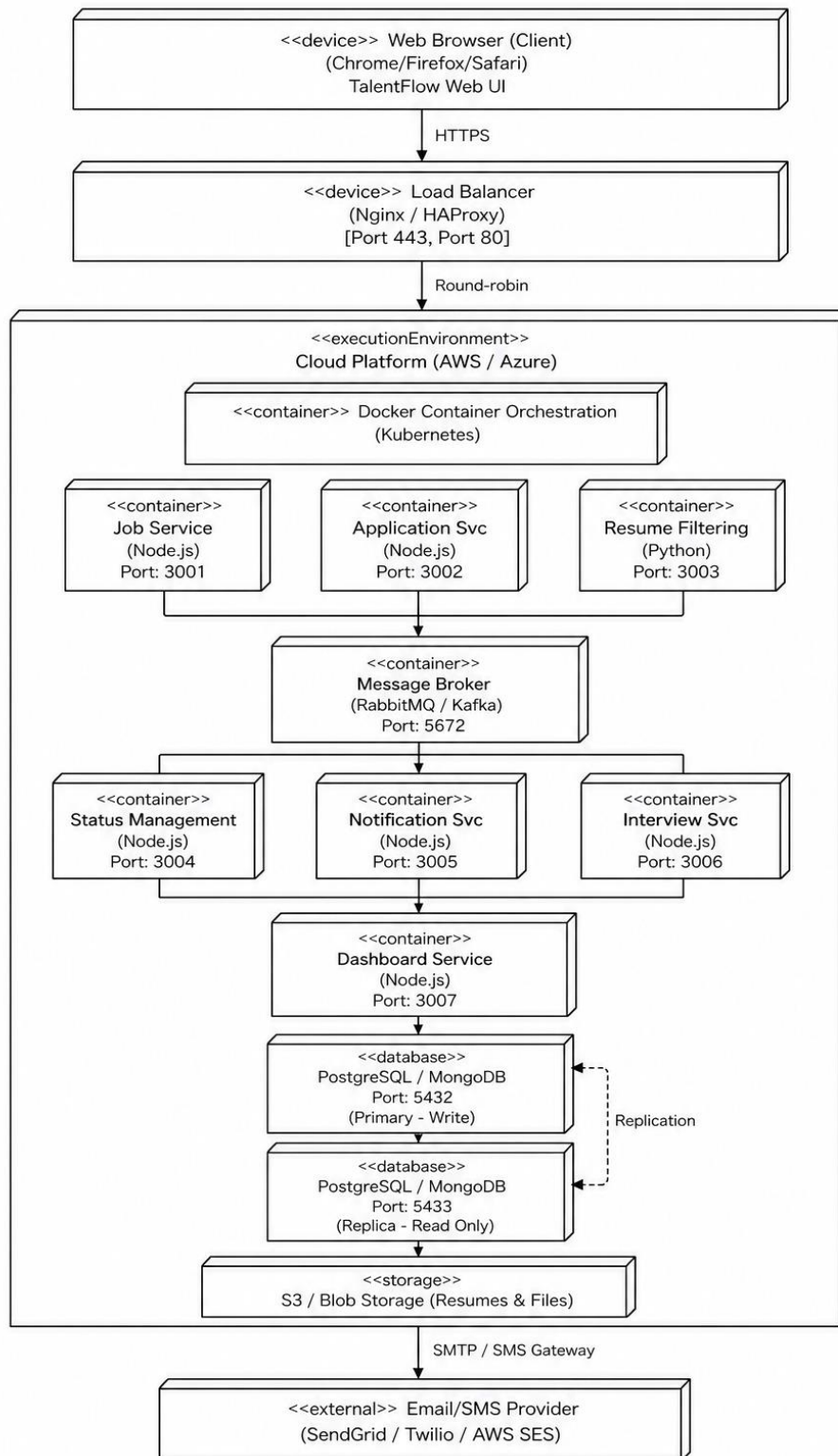


Diagram 1.4 Deployment Diagram

The deployment diagram shows how the TalentFlow system is physically deployed in a cloud environment. Users (candidates and recruiters) access the system via a web browser and send their requests via a secure HTTPS connection. A load balancer helps distribute the load effectively even with an intensive workload.

Cloud Infrastructure:

TalentFlow is deployed in a cloud environment and consists of different services managed by Kubernetes. Core services include:

- **Job Service:** Manages job operations
- **Application Service:** Handles application submissions
- **Resume Filtering Service:** Processes and screens resume

Supporting Services:

- **Notification Service:** Manages email/SMS communications
- **Interview Service:** Handles interview scheduling
- **Status Management Service:** Updates and tracks application status
- **Dashboard Service:** Provides real-time analytics

Infrastructure Components:

- **Primary DB and Replica DB:** Ensure data availability and redundancy
- **Cloud Storage:** Stores resume files and other documents
- **Message Broker (RabbitMQ/Kafka):** Facilitates event-driven communication
- **Kubernetes:** Orchestrates containerized services

This architecture provides scalability and efficiency for the system with high user activity.

6.0 TECHNOLOGY STACK

Architecture Layer	Technology Used	Purpose
Presentation Layer	HTML, CSS, JavaScript	User Interface for Candidates and Recruiters
Frontend Framework	React / Bookstrap	Responsive UI Components
Frontend Hosting	Vercel	Frontend Deployment
Application Layer	Node.js, Express.js	Business Logic Processing and API Management
Message Broker	RabbitMQ / Apache Kafka	Event Distribution and Communication
Security Component	JSON Web Tokens (JWT), Middleware	Authentication & Authorization
API Communication	REST API	Communication Between Frontend and Backend
Data Layer	MongoDB	Data Storage
Cloud Backend Hosting	Render	Backend Services Deployment
Container Orchestration	Kubernetes	Service Management and Scaling
Notification Service	AWS SES / Twilio	Email and SMS Notifications
File Storage	AWS S3 / Cloud Storage	Resume and Document Storage
Load Balancing	AWS ELB / Nginx	Traffic Distribution
Version Control	GitHub	Source Code Management

7.0 PROTOTYPE IMPLEMENTATION

The TalentFlow prototype was developed to demonstrate the core functionalities of the proposed Event-Driven Job Recruitment and Application Processing System. The prototype focuses on implementing a complete event-driven workflow to showcase the architectural principles.

The prototype includes:

7.1 Job Management Module

The Job Management Module was developed to allow recruiters to create and manage job postings. When a job is created, the system automatically triggers events that update the job board and notify relevant services.

Features Implemented:

- Job Creation by Recruiters
- Job Listing Display for Candidates
- JobPosted Event Publishing
- Automatic Job Board Updates

7.2 Application Management Module

The Application Management Module enables candidates to submit job applications with their resumes. Upon submission, the system automatically sends confirmation messages to candidates and triggers the resume screening process.

Features Implemented:

- Application Submission with Resume Upload
- Automatic Confirmation Messages
- ApplicationSubmitted Event Publishing
- Application Data Storage

7.3 Resume Filtering Module

The Resume Filtering Module automatically screens applicant resumes based on predefined criteria. This module consumes ApplicationSubmitted events and produces ResumeScreened events with filtering results.

Features Implemented:

- Automated Resume Screening
- Skills and Experience Matching
- Qualification Status Determination
- ResumeScreened Event Publishing

7.4 Status Management Module

The Status Management Module allows recruiters to update candidate status and automatically notifies candidates of changes. The module maintains a history of all status changes.

Features Implemented:

- Status Update by Recruiters
- Automatic Candidate Notifications
- Status History Tracking
- StatusUpdated Event Publishing

7.5 Notification Service Module

The Notification Service Module sends email and SMS notifications to candidates and recruiters based on events triggered throughout the recruitment process.

Features Implemented:

- Application Confirmation Emails
- Status Update Notifications
- Interview Invitations
- Reminder Notifications

The frontend interface was deployed using Vercel, while the backend services were hosted on Render. RabbitMQ was implemented as the message broker to handle event distribution between services.

8.0 ARCHITECTURE EVALUATION

The TalentFlow architecture was evaluated based on several quality attributes.

8.1 Scalability

The event-based architecture basically lets the system stretch out, like horizontally, whenever the user count jumps up. Each service can be tuned on its own, according to what demand is doing at the moment. So, for instance, the Resume Filter Service might run with several instances, all handling candidate applications in parallel during those peak hiring weeks. And because cloud hosting platforms are used, the underlying resources can be amplified when needed, without that heavy slowdown, or so the idea goes.

8.2 Maintainability

By separating concerns with microservices, the whole system gets way easier to maintain in practice. Each service kind of stays with a single responsibility so changes are less risky, and you can update one component without it messing up the rest. Also, the async communication via events helps a lot, because new services can show up without you having to rewrite or touch existing things. Like if later you want a new Analytics Service, you can just plug it in by subscribing to the right events, instead of doing invasive edits everywhere.

8.4 Performance

The system meets the performance requirements by handling events asynchronously, and honestly it feels pretty smooth because apps get an immediate confirmation. After that, notifications get pushed in real time, so it is not like you wait around. The whole event-driven design also supports parallel processing. That means the system can juggle several applications at once, without obvious stalls or delays.

8.5 Security

Authentication is based on JSON Web Tokens (JWT), and backend middleware adds another layer so only authorized users can reach protected resources. For permissions, role-based authorization stops anyone from doing recruiter functions they shouldn't. Also, candidate data and resumes are stored safely. Access controls are in place, so the right people can view things and the rest cannot.

8.6 Availability

Using cloud deployment via Vercel and Render helps keep system availability high. With Kubernetes in the mix, the platform is designed for automatic recovery when failures happen, plus load balancing across service instances, so the whole thing stays responsive. Overall, the system aims for 99.9% uptime, to keep continuous recruitment operations running without too much interruption

8.7 Responsiveness

The system gives candidates real-time updates across the entire recruitment process, not just at one checkpoint. Whenever a candidate status changes, they get immediate notifications, no long waiting around. Recruiters also get a dashboard with current metrics, which supports day to day decision-making in a more informed way.

9.0 TEAM CONTRIBUTIONS

Name	Role	Core System Contributions	Architectural Execution
Nurul Iffah Sabrina Binti Ahmad Shukri	System Architect & Designer	• Designed system structure and architecture • Created UML diagrams and architectural documentation • Decided on Event-Driven Architecture pattern • Designed the event flow and publish-subscribe model • Documented system components and their interactions	Event Architecture Design & Documentation
Izz Dhia Damia Binti Mat Zaidi	Backend & Integration Lead	• Developed backend system using Node.js and Express • Managed database design and integration • Handled API development and integration • Implemented message broker (RabbitMQ) integration • Developed core services and event handlers • Managed deployment on cloud platforms	Data Layer Architecture & Cloud DevOps Orchestration

10.0 CONCLUSION AND REFLECTION

TalentFlow was successfully developed as an Event-Driven Job Recruitment and Application Processing System, made to automate and streamline the hiring process for contemporary organizations. In practice, by implementing an Event-Driven Architecture (EDA) that uses a publish-subscribe model and RabbitMQ as the message broker, the system manages to tackle the main issues we usually see in classic recruitment flows, like longer hiring cycles, weak candidate communication, frequent mistakes, and a lot of extra pressure on HR teams. Overall, the architecture shows that asynchronous event processing helps with real-time responsiveness. Candidates can get confirmation quickly, along with status updates that feel more immediate, while recruiters can rely on automated resume screening, a less complicated interview planning step and also dashboard analytics that are detailed enough to support hiring decisions based on actual data. Another part that really stands out is the loose coupling between services. Because of that, the system stays scalable and manageable, and each component can be modified or scaled on its own without causing big disruptions to the entire operation. During this project, the team collected hands-on knowledge in event driven system design, message broker integration, microservices development and cloud deployment, and honestly it also deepened our understanding of how current architectural patterns can be applied to solve real world business problems. The work also emphasized how reliability mechanisms matter, such as message persistence and consumer acknowledgments, because they help ensure that information does not get lost even if failures happen. In conclusion, TalentFlow successfully demonstrates how Event-Driven Architecture principles can be leveraged to build an efficient, scalable, and responsive recruitment platform that meets the needs of conte